Novapolis Résilience

Correspondance entre le sujet et le code

Projet intégrateur – Algorithmique et théorie des graphes – ING-APP1-BDML

Objet. Pour chaque exigence de code du sujet, ce document indique le fichier et la fonction qui la réalisent, la manière dont elle est réalisée, et une commande ou un test qui permet de la vérifier. La colonne « Réf. » reprend la numérotation du sujet. Les livrables hors code (rapport, démonstration, relevé de contributions) ne sont pas traités ici.

Pour vérifier. Les commandes se lancent à la racine du dépôt, une fois l'environnement Python activé (voir le README). Toutes les sorties proposées vont dans `out/`, et les jeux livrés ne sont pas modifiés.

Installation	`pip install -e ".[dev]"`
Tous les tests (151)	`pytest`
Un seul test	`pytest -k nom_du_test`

2. Réseaux et données

Réf.	Exigence du sujet	Fichier et fonctions	Réalisation	Vérification
2.1	Réseau routier : graphe orienté pondéré, avec temps normal, temps de pointe, fermeture ou travaux ; type de voie et capacité à titre informatif.	`novapolis/graph/digraph.py` `DiGraph`, `arc_weight`, `build_road_graph` `novapolis/model.py` `Arc`, `FACTEUR_TRAVAUX`	Listes d'adjacence. Le poids est `temps_normal` ou `temps_pointe` selon la période. Un arc `ferme` est retiré ; un arc `travaux` voit son temps multiplié par 1,5 (hypothèse documentée). `type_voie` et `capacite` ne sont pas utilisés par les algorithmes.	Tests : `test_travaux_et_pointe`
2.1	Temps non négatifs dans les scénarios métier ; coûts négatifs réservés au jeu de test, sans les présenter comme des durées réalistes.	`novapolis/io/validator.py` `validate_tables` `novapolis/algorithms/shortest_paths.py` `dijkstra` `novapolis/cli.py` `negcycle`	Un temps négatif dans `routes.csv` est une erreur, sauf si `meta.json` contient `allow_negative` (jeux du module 3 uniquement). Dijkstra refuse les poids négatifs. La commande `negcycle` avertit que ces poids sont artificiels.	Tests : `test_negatif_autorise_seulement_en_test`, `test_poids_negatifs_interdits_hors_m3`, `test_dijkstra_refuse_poids_negatifs`
2.2	Réseau de fibre : graphe non orienté pondéré par le coût ; certaines arêtes interdites ou indisponibles.	`novapolis/graph/ugraph.py` `UGraph`, `build_fibre_graph`	Liste d'arêtes et listes d'adjacence. Coût en k€. Seuls les tronçons `disponible` sont candidats ; `interdit` et `panne` sont exclus.	`novapolis mst --data data/novapolis --algo both` Tests : `test_kruskal_prim_mini`
2.3	Fichiers CSV documentés : sites, routes, fibre, demandes, scénarios.	`data/SCHEMA.md` `novapolis/io/reader.py` `read_texts`, `parse_texts`, `tables_to_dataset`, `load_dataset` `novapolis/io/writer.py` `write_dataset`	Chaque jeu est un dossier de 5 CSV et un `meta.json`. Chaque champ est décrit dans `SCHEMA.md`.	Lecture du fichier
2.3	Fournir un validateur ; identifiants uniques ; arcs et arêtes vers un site inexistant rejetés avec un message explicite.	`novapolis/io/validator.py` `validate_tables`, `ValidationReport`	Chaque message cite le fichier, la ligne, la colonne et la valeur. Exemples : « Identifiant dupliqué », « Site inexistant … : arc rejeté ». Une erreur bloque le chargement.	`novapolis validate --data data/novapolis` Tests : `test_validateur_cas_d_erreur`, `test_validateur_avertissements`
2.3	Les fichiers de scénario ne modifient pas les données originales.	`novapolis/graph/scenario.py` `apply_actions`, `apply_scenario`	Le scénario s'applique sur une copie profonde (`deepcopy`) du jeu chargé.	Tests : `test_scenarios_sans_effet_sur_la_base`, `test_hash_fichiers_inchanges`

Réf.	Exigence du sujet	Fichier et fonctions	Réalisation	Vérification
2.4	Un petit graphe vérifiable à la main.	data/mini/ data/mini/EXPECTED.md tests/test_mini_expected.py	8 sites. Les résultats calculés à la main dans EXPECTED.md sont comparés à ceux du code.	pytest tests/test_mini_expected.py
2.4	Un jeu réaliste documenté d'au moins 100 sites.	data/novapolis/ data/novapolis/meta.json novapolis/generator.py realiste	150 sites (6 casernes, 4 hôpitaux, 12 écoles, 5 centres télécom), 40 demandes, 6 scénarios, graine 42.	novapolis validate --data data/novapolis Tests : test_realiste_conforme_au_sujet
2.4	Un jeu généré d'au moins 1 000 sommets.	data/generes/ novapolis/generator.py generate, preset_params, standard_datasets	1 000, 2 000 et 5 000 sommets, chacun en trois densités (degré moyen visé : 3, 6 et 12).	novapolis data --racine out/data Tests : test_presets_valides_et_reproductibles
2.4	Données synthétiques clairement identifiées.	data/SCHEMA.md data/novapolis/meta.json novapolis/cli.py entete	Tous les jeux sont générés ; aucune donnée externe n'est utilisée. meta.json porte synthetique, et la CLI affiche le badge SYNTHÉTIQUE.	Lecture du fichier

3.1 Module 1 : diagnostic d'accessibilité

Réf.	Exigence du sujet	Fichier et fonctions	Réalisation	Vérification
3.1	Implémenter DFS et BFS.	novapolis/algorithms/traversal.py bfs, dfs	Versions itératives (deque, pile explicite), donc sans limite de récursion. Listes d'adjacence : O(V + E).	Tests : test_bfs_dfs_depuis_c1, test_parcours_longs_sans_recursion
3.1.1	Sites accessibles depuis une ou plusieurs casernes.	novapolis/algorithms/traversal.py bfs, dfs novapolis/services.py m1_access, resolve_sources novapolis/cli.py access	--sources accepte les casernes, les hôpitaux ou une liste d'identifiants ; --algo choisit BFS ou DFS.	novapolis access --data data/novapolis --sources casernes --scenario normal Tests : test_bfs_multi_source
3.1.2	Composantes connexes du réseau de fibre.	novapolis/algorithms/traversal.py connected_components, _composantes	BFS sur le graphe non orienté de la fibre.	novapolis access --data data/novapolis --sources casernes --scenario normal Tests : test_mst
3.1.3	Itinéraire minimisant le nombre d'arcs.	novapolis/algorithms/traversal.py min_hop_path, build_path novapolis/services.py m1_minhop	BFS arrêté dès que la cible est découverte. La CLI le compare au chemin le plus rapide donné par Dijkstra.	novapolis minhop --data data/mini --from C2 --to H1 Tests : test_min_hop
3.1.4	Détecter les cycles orientés.	novapolis/algorithms/traversal.py find_directed_cycle	DFS à trois couleurs : un arc vers un sommet gris ferme un circuit, qui est renvoyé.	novapolis cycles --data data/mini Tests : test_connexites
3.1.5	Signaler les sites isolés ou non atteignables.	novapolis/algorithms/traversal.py isolated_sites novapolis/services.py m1_access	Isolé : degré entrant et sortant nuls. Non atteignable : non visité depuis les sources (non_atteignables).	novapolis access --data data/novapolis --sources casernes --scenario normal Tests : test_bfs_multi_source

Réf.	Exigence du sujet	Fichier et fonctions	Réalisation	Vérification

3.1	Ne pas confondre connexité ordinaire, accessibilité depuis une source et connexité forte.	novapolis/algorithms/traversal.py
		`bfs`, `weak_components`, `tarjan_scc`
		novapolis/cli.py
		`access`
			Trois calculs distincts : parcours depuis les sources, composantes sans tenir compte du sens, Tarjan. La sortie de la CLI indique la notion utilisée pour chaque indicateur.	`novapolis access --data data/novapolis --mode scc`
				Tests : `test_connexites`, `test_scc_et_faible`

3.2 Module 2 : itinéraires des secours

Réf.	Exigence du sujet	Fichier et fonctions	Réalisation	Vérification

3.2	Pour chaque demande, itinéraire depuis la caserne retenue jusqu'au site.	novapolis/indicators.py
		`request_routes`
		novapolis/services.py
		`m2_routes`
		novapolis/cli.py
		`routes`
			Un Dijkstra multi-source depuis toutes les casernes : chaque site hérite de la caserne qui l'atteint le plus vite.	`novapolis routes --data data/novapolis --scenario fermeture_axe`
				Tests : `test_demandes_par_scenario`

3.2.1	Implémenter Bellman–Ford et Floyd–Warshall.	novapolis/algorithms/shortest_paths.py `bellman_ford`, `floyd_warshall`	Bellman–Ford : liste d'arcs, $\|V\|-1$ passes au plus, arrêt anticipé. Floyd–Warshall : matrice $V \times V$, triple boucle, matrice des successeurs.	Tests : `test_trois_algos_identiques`
3.2.2	Ajouter Dijkstra pour les poids non négatifs.	novapolis/algorithms/shortest_paths.py `dijkstra`	Tas binaire `heapq` avec suppression paresseuse des entrées périmées.	Tests : `test_dijkstra_distances`

3.2.3	Reconstruire le chemin ; indiquer sa durée, son nombre d'arcs et les fermetures appliquées.	novapolis/algorithms/shortest_paths.py
		`path_from_parent`, `path_from_next`
		novapolis/indicators.py
		`request_routes`
		novapolis/cli.py
		`route`
			Reconstruction par les prédécesseurs ou par la matrice des successeurs. Champs `chemin`, `apres`, `arcs`, `fermetures_sur_chemin_initial`, `fermetures_proches`.	`novapolis route --data data/novapolis --demande D-02 --algo all --scenario fermeture_axe`
				Tests : `test_demandes_par_scenario`

3.2.4	Comparer les résultats des algorithmes sur les mêmes instances.	novapolis/services.py `m2_route`, `_algo_run`	`--algo all` exécute les trois algorithmes sur le même graphe et compare les distances (champ `identiques`).	`novapolis route --data data/novapolis --demande D-02 --algo all --scenario fermeture_axe` Tests : `test_trois_algos_identiques`, `test_plus_courts_chemins`

3.2.5	Scénario de fermeture : désactiver des arcs, recalculer, mesurer la dégradation.	novapolis/graph/scenario.py
		`apply_scenario`
		novapolis/indicators.py
		`request_routes`
		data/novapolis/scenarios.csv
			Les arcs fermés sont retirés de la copie, puis les itinéraires sont recalculés et comparés à la situation de base.	`novapolis routes --data data/novapolis --scenario fermeture_axe`
				Tests : `test_demandes_par_scenario`

3.2	Par demande : coût avant et après, retard absolu et relatif, « accessible » ou « inaccessible », chemin retenu.	novapolis/indicators.py `request_routes`	Champs `avant`, `apres`, `retard`, `retard_pct`, `statut`, `chemin`. L'option `--export` les écrit en CSV ou JSON.	`novapolis routes --data data/novapolis --scenario fermeture_axe --export out/routes.csv` Tests : `test_demandes_par_scenario`
3.2	Discuter les limites des données, des coûts et du modèle.	README.md	Temps statiques, capacité des casernes et incidents simultanés ignorés, facteur travaux supposé, coûts de fibre synthétiques.	Lecture du fichier

3.3 Module 3 : détection des circuits absorbants

Réf.	Exigence du sujet	Fichier et fonctions	Réalisation	Vérification
3.3.1	Graphe de test avec un circuit absorbant accessible depuis la source.	data/negatif_accessible/ novapolis/generator.py test_m3_a	Circuit A → B → C → A de poids −1, accessible depuis S.	novapolis negcycle --data data/negatif_accessible --source S --algo all Tests : test_graphe_a_accessible
3.3.2	Graphe avec un circuit absorbant non accessible depuis la source.	data/negatif_inaccessible/ novapolis/generator.py test_m3_b	Circuit C → D → E → C de poids −1, non accessible depuis S : les distances restent finies ou +∞.	novapolis negcycle --data data/negatif_inaccessible --source S --algo all Tests : test_graphe_b_non_accessible
3.3.3	Détecter si un circuit absorbant existe et préciser s'il est accessible depuis la source.	novapolis/algorithms/negative_cycles.py negcycle_bellman_ford, negcycle_floyd, detect_global_cycle	Bellman–Ford : un arc encore relâchable après |V|−1 passes ; l'existence globale se teste en partant de tous les sommets à 0. Floyd–Warshall : d[k][k] < 0, accessible si d[s][k] est fini.	novapolis negcycle --data data/negatif_accessible --source S --algo all Tests : test_graphe_a_accessible, test_graphe_b_depuis_c, test_sans_circuit, test_boucle_negative
3.3.4	Ne pas présenter comme finie une distance qui peut diminuer indéfiniment.	novapolis/algorithms/negative_cycles.py _propager_moins_inf novapolis/services.py clean novapolis/cli.py f	Tout sommet atteignable depuis un sommet marqué passe à −∞, encodé -inf puis affiché « −∞ ».	novapolis negcycle --data data/negatif_accessible --source S --algo all Tests : test_graphe_a_accessible

3.4 Module 4 : conception du réseau de fibre

Réf.	Exigence du sujet	Fichier et fonctions	Réalisation	Vérification
3.4	Implémenter Kruskal et Prim.	novapolis/algorithms/mst.py kruskal, prim novapolis/algorithms/union_find.py UnionFind	Kruskal : tri des arêtes et Union–Find (compression de chemin, union par rang). Prim : tas binaire.	Tests : test_kruskal_prim_mini, test_mst, test_union_find
3.4.1	Arbre couvrant minimal, ou forêt couvrante si le graphe est non connexe.	novapolis/algorithms/mst.py kruskal, prim	Kruskal produit naturellement une forêt. Prim redémarre depuis le prochain sommet non couvert quand le tas est vide.	novapolis mst --data data/novapolis --algo both Tests : test_kruskal_prim_mini
3.4.2	Afficher les tronçons choisis, le coût total et les composantes non raccordées.	novapolis/services.py m4_mst novapolis/cli.py mst	Champs aretes, cout et non_raccordes.	novapolis mst --data data/novapolis --algo both
3.4.3	Comparer le résultat et le coût total des deux algorithmes.	novapolis/services.py m4_mst	Champs cout_identique, meme_ensemble, diff_kruskal, diff_prim.	novapolis mst --data data/novapolis --algo both Tests : test_mst
3.4.4	Traiter les coûts égaux et expliquer pourquoi plusieurs arbres minimaux peuvent exister.	novapolis/algorithms/mst.py tie_analysis	Une arête hors de l'arbre de même coût que l'arête maximale du cycle qu'elle crée peut être échangée sans changer le coût total. La CLI affiche un tel échange.	novapolis mst --data data/novapolis --algo both Tests : test_kruskal_prim_mini

Réf.	Exigence du sujet	Fichier et fonctions	Réalisation	Vérification

3.4.5	Panne : retirer un tronçon de l'arbre et calculer le raccordement le moins coûteux disponible dans le graphe initial.	novapolis/algorithms/mst.py
		`repair_after_failure`, `split_tree`
		novapolis/services.py
		`m4_panne`
		novapolis/cli.py
		`mst_panne`
			Le retrait coupe l'arbre en deux ; on retient l'arête disponible la moins chère qui relie les deux morceaux, hors tronçon en panne.	`novapolis mst-panne --data data/mini --edge H1-Q2`
				Tests : `test_panne_mini`

3.5 Module 5 : résilience et scénarios

Réf.	Exigence du sujet	Fichier et fonctions	Réalisation	Vérification

3.5	Évaluer au moins trois scénarios : situation normale, fermeture d'un axe, panne de fibre ou zone de travaux.	data/novapolis/scenarios.csv
		novapolis/services.py
		`m5_scenarios`
		novapolis/cli.py
		`scenarios`
			Six scénarios sont fournis. `normal`, `fermeture_axe`, `panne_fibre` et `zone_travaux` couvrent les trois situations demandées ; `heure_pointe` et `crue_nova` s'y ajoutent.	`novapolis scenarios --data data/novapolis --all`

3.5	Indicateur : nombre de demandes devenues inaccessibles.	novapolis/indicators.py `scenario_indicators`	Champ `demandes_devenues_inaccessibles`.	`novapolis scenarios --data data/novapolis --all` Tests : `test_indicateurs_mini`
3.5	Indicateur : augmentation moyenne et maximale du temps de trajet.	novapolis/indicators.py `scenario_indicators`	Champs `hausse_moyenne` et `hausse_max`.	`novapolis scenarios --data data/novapolis --all` Tests : `test_indicateurs_mini`
3.5	Indicateur : nombre de composantes du réseau de fibre.	novapolis/indicators.py `fibre_indicators`	Champs `composantes_fibre_avant_reparation` et `composantes_fibre`.	`novapolis scenarios --data data/novapolis --all` Tests : `test_indicateurs_mini`

3.5	Indicateur : coût de la réparation ou du raccordement.	novapolis/indicators.py
		`fibre_indicators`
		novapolis/algorithms/mst.py
		`repair_forest`
			L'arbre de base, privé des tronçons perdus, est complété au moindre coût. Champ `cout_reparation`.	`novapolis scenarios --data data/novapolis --all`
				Tests : `test_indicateurs_mini`

3.5	Indicateur : proportion des sites sensibles restant accessibles.	novapolis/indicators.py
		`sensitive_share`
		novapolis/model.py
		`SENSIBLE_TYPES`
			Hôpitaux, casernes, écoles et centres télécom atteignables depuis une caserne. Champ `part_sensibles_accessibles`.	`novapolis scenarios --data data/novapolis --all`
				Tests : `test_indicateurs_mini`

3.5	Perturbations appliquées sur une copie ; scénario de référence inchangé.	novapolis/graph/scenario.py
		`apply_actions`
		novapolis/io/writer.py
		`dataset_hash`
		novapolis/indicators.py
		`scenario_indicators`
			Copie profonde, puis empreinte SHA-256 de la base comparée avant et après (`base_inchangee`), affichée par la CLI.	`novapolis scenarios --data data/novapolis --all`
				Tests : `test_scenarios_sans_effet_sur_la_base`, `test_hash_fichiers_inchanges`

4. Architecture logicielle

Réf.	Exigence du sujet	Fichier et fonctions	Réalisation	Vérification
4	Lecture, validation et écriture des données.	novapolis/io/reader.py novapolis/io/validator.py novapolis/io/writer.py	Paquet `io`.	Lecture du fichier
4	Structure de graphe et application des scénarios.	novapolis/graph/digraph.py novapolis/graph/ugraph.py novapolis/graph/scenario.py	Paquet `graph`.	Lecture du fichier
4	Parcours et algorithmes de plus courts chemins.	novapolis/algorithms/traversal.py novapolis/algorithms/shortest_paths.py novapolis/algorithms/negative_cycles.py	Paquet `algorithms`.	Lecture du fichier
4	Algorithmes d'arbres couvrants.	novapolis/algorithms/mst.py novapolis/algorithms/union_find.py	Paquet `algorithms`.	Lecture du fichier
4	Calcul des indicateurs et benchmarks (facultatif).	novapolis/indicators.py novapolis/bench.py	Modules dédiés.	Lecture du fichier
4	Interface en ligne de commande ; sélection d'un fichier, d'un scénario et d'une opération.	novapolis/cli.py `app`	`--data` choisit le jeu de données, `--scenario` le scénario, la sous-commande l'opération.	`novapolis route --data data/novapolis --demande D-02 --algo all --scenario fermeture_axe`
4	Tests automatisés (facultatif).	tests/	151 tests.	`pytest`
4	Exemple d'utilisation dans le README ; choix des structures de données justifiés.	README.md	Python. Listes d'adjacence, liste d'arêtes, matrice, Union–Find, tas binaire et `deque` : chaque choix est justifié dans le README.	Lecture du fichier

5. Expériences et analyse

Réf.	Exigence du sujet	Fichier et fonctions	Réalisation	Vérification
5.1	Mesurer DFS, BFS, Kruskal, Prim, Dijkstra et Bellman–Ford sur plusieurs tailles et densités.	novapolis/bench.py `run_bench`, `bench_graph`, `ALGOS`	De 250 à 5 000 sommets ; densités faible, moyenne et forte.	`novapolis bench --sizes 250,500,1000,2000,5000 --reps 5 --seed 42 --out out/bench.csv`
5	Mêmes graphes pour tous les algorithmes ; chargement, affichage et génération exclus du temps mesuré.	novapolis/bench.py `bench_graph`, `_mesurer`	Un graphe par couple (taille, densité). `perf_counter` n'entoure que l'appel de l'algorithme. Kruskal et Prim utilisent le réseau de fibre de ce même jeu.	Lecture du fichier

5.2	Mesurer Floyd–Warshall sur des tailles plus petites et expliquer ce qui limite son usage.	novapolis/bench.py
		`FLOYD_TAILLES`
		novapolis/services.py
		`FLOYD_MAX_N`
		scripts/make_figures.py
		reports/figures/fig3_floyd_warshall.png
			$n \leq 400$. La figure 3 ajuste $c \cdot V^3$ et extrapole le temps et la mémoire à $n = 5\,000$.	`novapolis bench --sizes 250,500,1000,2000,5000 --reps 5 --seed 42 --out out/bench.csv`

5.3	Répéter chaque expérience au moins cinq fois et présenter la moyenne.	novapolis/bench.py `run_bench`, `_mesurer`	`reps` est forcé à 5 au minimum. Le CSV contient moyenne, écart-type, minimum et maximum.	`novapolis bench --sizes 250,500,1000,2000,5000 --reps 5 --seed 42 --out out/bench.csv`

Réf.	Exigence du sujet	Fichier et fonctions	Réalisation	Vérification
5.4	Fixer et documenter les graines aléatoires.	novapolis/bench.py run_bench novapolis/generator.py preset_params data/novapolis/meta.json	Graine 42, notée dans la colonne graine du CSV de benchmarks et dans le meta.json des jeux aléatoires (novapolis et generes/). mini et les deux jeux du module 3 sont fixes, sans tirage aléatoire.	Lecture du fichier
5.5	Tracer au moins trois graphiques lisibles, avec unités et légendes.	scripts/make_figures.py main reports/figures/	Quatre figures PNG : temps selon la taille, temps selon la densité, Floyd–Warshall, rapport mesure sur théorie.	python scripts/make_figures.py out/ bench.csv --out out/figures
5.6	Comparer les mesures aux complexités théoriques et discuter les écarts.	novapolis/bench.py theorique, analyse, commentaire scripts/make_figures.py	Pente log-log mesurée comparée à la pente théorique, constante ajustée, commentaire de chaque écart. Figures 1 et 4.	novapolis bench --sizes 250,500,1000,2000,5000 --reps 5 --seed 42 --out out/bench.csv

6. Livrables de code

Réf.	Exigence du sujet	Fichier et fonctions	Réalisation	Vérification
6.1	Code source organisé en modules.	novapolis/	Voir la section 4.	Lecture du fichier
6.2	Données d'exemple, description des champs et générateur reproductible.	data/ data/SCHEMA.md novapolis/generator.py generate	novapolis data régénère les jeux à partir de leurs paramètres et graines. Avec --racine, le dossier data/ n'est pas modifié.	novapolis data --racine out/data Tests : test_presets_valides_et_reproductibles, test_graines_differentes
6.3	Tests automatisés couvrant les algorithmes et les cas limites (facultatif).	tests/	Jeu mini comparé au calcul à la main, 100 cas croisés avec networkx (utilisé uniquement dans les tests), cas limites, module 3, validateur, reproductibilité.	pytest Tests : test_graphe_vide, test_cible_inaccessible, test_tous_les_arcs_fermes
6.4	README d'installation et d'utilisation, avec commandes exécutables.	README.md	Installation, commandes par module, jeux de données, architecture, tests, benchmarks.	Lecture du fichier